

실험 전 보고서

LAB1 - 조합논리 10개

실험 주제 : LAB1 조합논리 10개 - 입력 조합 및 출력 관계 분석

실험 대상 : AND·OR·XOR 게이트, 전가산기, 4비트 감·가산기/크기 비교기,
4:1 MUX, 1:8 DeMUX, 8:3 Incoder, 3:8 Decoder,
7-Segment Decoder

작성일 : 2026년 09월 13일

검증 환경 : VS Code Vapor View, Vivado 2026.1

성명 : 권진혁

학과 : 전자전기컴퓨터공학부

학번 : 2023440007

1. AND·OR·XOR 논리 게이트

· 목적

Verilog HDL을 이용해 2입력 AND, OR, XOR 논리 게이트를 설계하고 TB를 통해 가능한 입력 조합에 대한 출력을 검증한다. 출력을 Vapor View로 확인하여 실제 파형이 진리표와 일치하는지 분석한다.

· 입출력 구성

설계한 논리 게이트 회로는 두 개의 1비트 입력 a, b와 세 개의 1비트 출력 x, y, z로 구성하였다. a와 b는 논리 입력이고 x, y, z는 각각 AND, OR, XOR 연산 결과에 해당한다.

$x = a \cdot b$, $y = a + b$, $z = a \oplus b$ 의 논리식을 지닌다. AND 연산은 입력 모두 1인 경우에만 1을 출력하고, OR은 두 입력 중 하나 이상이 1이면 1을 출력하며 XOR은 두 입력이 서로 다른 경우에만 1을 출력한다.

· 진리표

a	b	x	y	z
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

입력이 00인 경우 모든 출력은 0이다. 입력이 01 or 10인 경우 AND 출력은 0이고 OR, XOR은 1이다. 입력이 11인 경우 AND, OR 출력은 1이지만, 두 입력이 서로 같으므로 XOR 출력은 0이다. 이 진리표를 시뮬레이션 결과의 기준으로 활용하였다.

· RTL 설계

RTL 파일의 경로는 'src/logic_gate.v'이다. 설계 모듈의 이름은 'logic_gate'이며 코드는 다음과 같다.

```
1 // PDF의 회로 코드를 여기에 직접 작성하고 저장하세요.
2 // 파일명을 바꾸거나 하위 모듈을 추가하면 simulation.json의 sources도 수정하세요.
3
4 module logic_gate (
5     input wire a,
6     input wire b,
7     output wire x,
8     output wire y,
9     output wire z
10 );
11
12     assign x = a & b; // AND gate
13     assign y = a | b; // OR gate
14     assign z = a ^ b; // XOR gate
15
16
17 endmodule
18
```

· 사용 도구 및 버전

```
PS C:\Users\kjh36\Documents\Verilog\lab1_01_logic_gates> git --version
git version 2.55.0.windows.5
PS C:\Users\kjh36\Documents\Verilog\lab1_01_logic_gates> python --version
Python 3.14.7
PS C:\Users\kjh36\Documents\Verilog\lab1_01_logic_gates> iverilog -V
Icarus Verilog version 14.0 (devel) ()
```

```
PS C:\Users\kjh36\Documents\Verilog\lab1_01_logic_gates> vvp -V
Icarus Verilog runtime version 14.0 (devel) ()
```

```
PS C:\Users\kjh36\Documents\Verilog\lab1_01_logic_gates> code --version
1.137.0
645f29cc3176500b4b5762ba887cf2a7f0ffdf2c
arm64
```

· 코드 커밋

```
PS C:\Users\kjh36\Documents\Verilog\lab1_01_logic_gates> git rev-parse HEAD
a248b802aa2dec8def8caaf5175f12fc005f8f5c
PS C:\Users\kjh36\Documents\Verilog\lab1_01_logic_gates> git branch --show-current
main
```

· TB 구성 및 예상 결과

```
1 timescale 1ns/1ps
2
3 module tb_logic_gate_modern;
4
5     reg a,b; wire x,y,z; logic_gate dut(a,b,b,x,y,y,z,z);
6     integer n; checked=0;
7     reg [2:0] expected;
8     initial begin
9         $dumpfile("wave.vcd"); $dumpvars(0,tb_logic_gate_modern);
10
11         for (n=0; n<4; n=n+1) begin
12             {a,b} = n;
13             expected = {a&b, a|b, a^b};
14             #10;
15             if ({x,y,z} != expected)
16                 $fatal(1, "Fail logic_gate vector=%0d expected=%h actual=%h",n,ex
17                 checked=checked+1;
18         end
19     end
```

```

20
21     if (checked != 4) $fatal(1, "Incomplete test");
22     $display("LAB1_PASS logic_gate cases=%0d", checked);
23     $finish;
24 end
25 initial begin #100000; $fatal(1, "Watchdog"); end
26
27 endmodule

```

```

1
2 {
3   "sources": ["src/logic_gate.v"],
4   "testbench": "sim/tb_logic_gate_modern.sv",
5   "simulation_top": "tb_logic_gate_modern"
6 }

```

TB에서 입력은 다음 순서로 인가하였다.
 “00 -> 01 -> 10 -> 11”. 각 입력 조합은
 10ns 동안 유지되며 예상 결과는 다음과 같다.

순서	입력	예상값	구간
1	00	000	0~10ns
2	01	011	10~20ns
3	10	011	20~30ns
4	11	110	30~40ns

TB 첫 부분을 통해 기본 시간 단위는 1ns,
 정밀도는 1ps로 설정하고 ‘#10’으로 10ns의
 지연을 설정하였다. 각 입력이 적용된 이후
 10ns를 기다려 출력을 확인하도록 구성하였
 다. 모든 조합을 검사하는데 40ns가 사용되
 며 모든 실제 출력이 기댓값과 일치하면
 ‘LAB1_PASS logic_gate cases=4’라는 문
 장을 출력하도록 하였다. 로그에서 종료시각은
 40000ps로 출력 이후 ‘\$finish’를 실행하여
 40ns에 정상 종료하도록 하였다. 또한, 정상
 적인 종료가 이루어지지 않을 경우를 검출하
 기 위해 40ns보다 충분히 긴 시간 뒤에 동
 작하는 ‘watchdog’을 별도로 구성하였다.

· 결과

02 Simulation Task를 실행한 결과
 iverilog를 이용한 컴파일과 vvp를 이용한
 시뮬레이션이 차례대로 수행되었다. 정상 실행
 결과 다음과 같은 PASS 문구와 정상 종
 료 시각을 확인하였다.

```

Python 3.14.7
RUN "C:\Program Files\Git\cmd\git.EXE" --version
git version 2.55.0.windows.5

RUN C:\iverilog\bin\iverilog.EXE -V
Icarus Verilog version 14.0 (devel) ()

Copyright (c) 2000-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

```

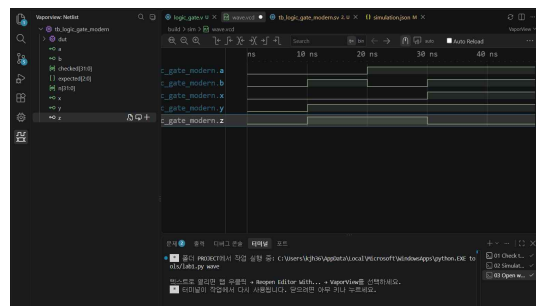
```

RUN C:\iverilog\bin\iverilog.EXE -g2012 -s1 C:\Users\kj36\Documents\Verilog\lab1_01_logic_gate
es\src -I C:\Users\kj36\Documents\Verilog\lab1_01_logic_gates\sim -s tb_logic_gate_modern -o C:\Us
ers\kj36\Documents\Verilog\lab1_01_logic_gates\build\sim\run-end8f1b21b894817b7f71a261dce1\sim.
vvp C:\Users\kj36\Documents\Verilog\lab1_01_logic_gates\src\logic_gate.v C:\Users\kj36\Documents\
Verilog\lab1_01_logic_gates\sim\tb_logic_gate_modern.sv -
run C:\iverilog\bin\vvp.EXE -M C:\Users\kj36\Documents\Verilog\lab1_01_logic_gates\build\sim\run-e
8d8f1b21b894817b7f71a261dce1\sim.vvp
VCD Info: dumpfile wave.vcd opened for output.
LAB1_PASS logic_gate cases=4
C:\Users\kj36\Documents\Verilog\lab1_01_logic_gates\sim\tb_logic_gate_modern.sv:23: $finish called
at 40000 (1ps)

시뮬레이션 실행 종료. 결과와-지속검사-회로으로 회로의 경우 결과를 확인하세요.
wave: C:\Users\kj36\Documents\Verilog\lab1_01_logic_gates\build\sim\wave.vcd
테이블이 작업에서 다시 사용됨니다. 닫으려면 아무 키나 누르세요.

```

단순히 ‘wave.vcd’가 생성된 것만으로 성
 공여부를 확정 짓는 것이 아닌 자기검사
 PASS 문구와 종료시각을 함께 확인하였다.
 또한, 시뮬레이션에서 생성된 파형 파일은
 다음과 같다.



구간별 파형을 분석해 보면 ‘0~10ns’ 구간
 에선 입력이 둘 다 0이기 때문에 x=0, y=0,
 z=0이 된다. ‘10~20ns’ 구간에선 a=0, b=1
 이기 때문에 x=0, y=1, z=1이 된다.
 ‘20~30ns’ 구간에선 a=1, b=0이기 때문에
 x=0, y=1, z=1이 된다. ‘30~40ns’에선 입력

이 둘 다 1이기 때문에 $x=1, y=1, z=0$ 이 된다. 이는 실제 파형을 분석함으로써 파악할 수 있는 내용이며 진리표와도 일치함을 알 수 있다.

· OR, XOR의 차이

OR와 XOR은 입력 00, 01, 10에서는 동일한 출력값을 가진다. (00, 0, 0), (01, 1, 1), (10, 1, 1). 그러나 11에서는 두 연산의 출력이 달라진다. OR는 1이 나오고, XOR는 0이 나온다. 즉, 두 논리합의 핵심적인 차이는 두 입력이 모두 1인 경우에 나타난다. OR은 포함적 논리합이므로 11에서도 1을 출력하지만, XOR는 배타적 논리합이므로 두 입력이 동시에 1인 경우를 제외하고 0을 출력한다. 이는 30~40ns 구간의 파형을 살펴 보아도 알 수 있다. 이 결과는 사전에 작성한 진리표와 일치하며, 작성한 RTL의 OR 및 XOR 연결이 올바르게 동작했음을 시사한다.

2. 전가산기 : 반가산기 두 개의 연결

· 목적

두 개의 Half Adder를 이용하여 Full Adder를 설계하고 Verilog HDL로 구현한다. 또한, TB를 작성하여 모든 입력 조합에 대한 기능을 검증하고 이를 VS Code를 이용한 시뮬레이션을 통해 이론값과 비교한다.

· 회로 이론

Half Adder의 입력은 A와 B로 이루어진다. 출력은 'Sum=A XOR B'와 'Carry=A

AND B'이다. Full Adder의 입력은 A, B, Cin으로 이루어지며 출력은 's'와 'Cout'이다. Half Adder는 1자리 이진수 2개(A, B)를 더하여 Sum과 Carry를 출력한다. Full Adder는 A, B와 하위 자리올림수인 Cin까지 총 3개의 입력을 더해 Sum과 Carry를 출력한다.

· 진리표

a	b	Cin	Cout	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

· RTL 설계

```
src > @ half_adder.v ...
1 module half_adder(
2     input a,
3     input b,
4     output s,
5     output c
6 );
7
8 assign s = a ^ b; // Sum output
9 assign c = a & b; // Carry output
10
11 endmodule
12
```

```

1 module full_adder(
2     input a,
3     input b,
4     input cin,
5     output s,
6     output cout
7 );
8
9 wire sum1;
10 wire carry1;
11 wire carry2;
12
13 half_adder HA1 (
14     .a(a),
15     .b(b),
16     .s(sum1),
17     .c(carry1)
18 );
19

```

Full Adder는 두 개의 Half Adder를 계층적으로 연결하여 구현하였다. 첫 번째 Half Adder에서 a, b를 더하여 sum1과 carry1를 생성하고, 두 번째 Half Adder에서 sum1과 Cin을 더하여 s와 carry2를 생성하였다. Cout은 carry1과 carry2를 OR 연산하여 생성하였다.

· 사용 도구 및 버전

```

PS C:\Users\kjh36\Documents\Verilog\lab1_01_logic_gates> git --version
git version 2.55.0.windows.5
PS C:\Users\kjh36\Documents\Verilog\lab1_01_logic_gates> python --version
Python 3.14.7
PS C:\Users\kjh36\Documents\Verilog\lab1_01_logic_gates> iverilog -V
Icarus Verilog version 14.0 (devel) ()

```

```

PS C:\Users\kjh36\Documents\Verilog\lab1_01_logic_gates> vvp -V
Icarus Verilog runtime version 14.0 (devel) ()

```

```

PS C:\Users\kjh36\Documents\Verilog\lab1_01_logic_gates> code --version
1.137.0
645f29cc3176500b4b5762ba887cf2a7f0ffdf2c
arm64

```

· 코드 커밋

```

PS C:\Users\kjh36\Documents\Verilog\lab1_02_full_adder> git branch --show-current
main
PS C:\Users\kjh36\Documents\Verilog\lab1_02_full_adder> git rev-parse HEAD
a248b02aa2dec8def8caaf5175f12fc005f8f5c
PS C:\Users\kjh36\Documents\Verilog\lab1_02_full_adder>

```

· TB 구성 및 예상 결과

```

1 `timescale 1ns/1ps
2
3 module tb_full_adder;
4
5     reg a,b,cin; wire s,cout; full_adder dut(a,b,b,cin; cin,s,cout; cout);
6     integer n,checked=0;
7     reg [1:0] expected;
8
9     initial begin
10         $dumpfile("wave.vcd");
11         $dumpvars(0,tb_full_adder);
12
13         for(n=0;n<8;n++) begin
14             {a,b,cin} = n;
15             expected = int'(a)+int'(b)+int'(cin);
16             #10;
17             if({cout,s} != expected)
18                 $fatal(1, "FAIL full adder vector=%0d expected=%0d actual=%0d", n, expected, {cout,s});
19             checked = checked + 1;
20         end
21

```

```

22         if(checked != 8) $fatal(1, "Incomplete test");
23         $display("LAB1_PASS full_adder cases=%0d", checked);
24         $finish;
25     end
26     initial begin
27         #100000;
28         $fatal(1, "Watchdog"); end
29
30 endmodule

```

예상 결과는 다음과 같다.

구간 (ns)	a	b	Cin	계산	Cout	S
0~10	0	0	0	0+0+0=0	0	0
10~20	0	0	1	0+0+1=1	1	0
20~30	0	1	0	0+1+0=1	1	0
30~40	0	1	1	0+1+1=2	0	1
40~50	1	0	0	1+0+0=1	1	0
50~60	1	0	1	1+0+1=2	0	1
60~70	1	1	0	1+1+0=2	0	1
70~80	1	1	1	1+1+1=3	1	1

· 입력 조합, 기댓값, 자동 비교 종료 조건

본 실험의 테스트벤치는 전가산기의 모든 입력 조합을 검증하도록 작성하였다. 입력

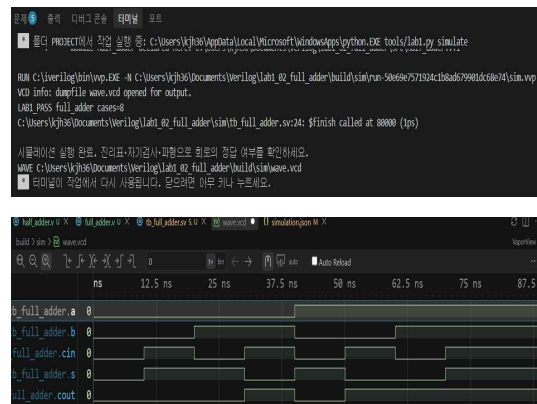
a, b, Cin에 대하여 000부터 111까지 총 8개의 조합을 10ns 간격으로 순차적으로 인가하였다. Full Adder에서는 모든 입력 조합을 검사하고 검사 횟수가 8회가 되도록 구성하도록 안내하고 있다. 각 입력 조합에 대해 기댓값은 입력 신호 a, b, Cin의 정수 덧셈 결과로부터 계산하였다. 출력 s는 덧셈 결과의 최하위 비트를 나타내고, 출력 Cout은 자리올림 비트를 나타낸다. TB는 DUT의 실제 출력값과 계산된 기댓값을 자동으로 비교한다. 만약 실제 출력과 기댓값이 일치하지 않으면 \$fatal 명령을 이용하여 시뮬레이션을 즉시 중단하고 오류 메시지를 출력하도록 구성하였다. 또한, != 비교 연산을 사용하여 X 또는 Z 상태까지 검출할 수 있도록 하였다. 모든 입력 조합에 대한 검사가 정상적으로 완료되면 검사 횟수를 확인한 후 'LAB1_PASS full_adder cases=8'를 출력하고 \$finish를 통해 시뮬레이션을 종료한다. 또한, 비정상적으로 시뮬레이션이 종료되지 않는 경우를 대비하여 watchdog을 두어 테스트가 무한정 실행되지 않도록 하였다.

· 결과

```
Python 3.14.7
RUN "C:\Program Files\Git\cmd\git.EXE" --version
git version 2.55.0.windows.5

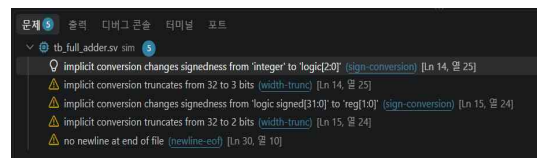
You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/sim/constraints/의 파일을 직접 작성하세요.
터미널이 작업에서 다시 사용됩니다. 닫으려면 아무 키나 누르세요.
```

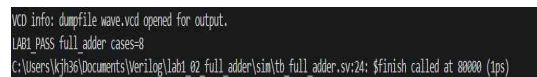


· 오류 검증

‘cout=carry1 | carry2’를 ‘cout=carry1 & carry2’로 변경했으면 아래 사진과 같은 오류가 발생한다.



이를 수정하여 다시 실행하면 정상적으로 작동한다.



이를 통해 TB가 회로 오류를 정상적으로 검출함을 확인하였다.

3. 4비트 가산기

· 목적

Verilog HDL을 이용하여 Adder_4bit를 설계하고, 시뮬레이션을 통해 결과를 검증하는 것이다. 0~15 범위의 4비트 이진수를 입력받아 덧셈을 수행하고, 결과를 5비트 형태로 출력한다.

· 입출력 포트, 비트 폭

포트명	방향	비트 폭	설명
a	In	4bit	1 st 피연산자
b	In	4bit	2 nd 피연산자
s	Out	4bit	덧셈 결과의 하위 4비트
cout	Out	1bit	Carry-out

· 예상표

a	b	결과	cout	s
0	0	0	0	0000
1	15	16	1	0000
15	15	30	1	1110

모든 결과인 256가지를 적는 것은 무리이므로 핵심 경계 조건 3가지를 기술하였다.

· 동작원리

가산기는 두 개의 4비트 입력 a, b를 더하여 5비트 결과를 생성한다. 결과값의 하위 4비트는 s에 저장되고, 최상위 비트는 cout으로 출력된다. 따라서 연산 결과가 15를 초과하면 cout이 1이 되며 자리올림 발생 여부를 나타낸다.

· 코드 커밋 & 도구 버전

```
PS C:\Users\kjh36\Documents\Verilog\lab1_03_adder4> git --version
git version 2.55.0.windows.5
PS C:\Users\kjh36\Documents\Verilog\lab1_03_adder4> python --version
Python 3.14.7
PS C:\Users\kjh36\Documents\Verilog\lab1_03_adder4> code --version
1.137.0
645f29cc3176500b4b5762ba887cf2a7f0ffdf2c
arm64

PS C:\Users\kjh36\Documents\Verilog\lab1_03_adder4> iverilog -V
Icarus Verilog version 14.0 (devel) ()

PS C:\Users\kjh36\Documents\Verilog\lab1_03_adder4> vvp -V
Icarus Verilog runtime version 14.0 (devel) ()

PS C:\Users\kjh36\Documents\Verilog\lab1_03_adder4> git branch --show-current
main
PS C:\Users\kjh36\Documents\Verilog\lab1_03_adder4> git rev-parse HEAD
a248b802aa2dec8def8caaf5175f12fc005f8f5c
```

· RTL & TB

```
adder_4bit.v 1.U X tb_adder_4bit.v 5.U X wavevcd simulation.json M X
src > @ adder_4bit.v > {} adder_4bit
1 module adder_4bit(input wire [3:0] a,b, output wire [3:0] s, output wire cout);
2   assign {cout,s} = {1'b0,a} + {1'b0,b};
3 endmodule
```

해당 RTL은 모듈 adder_4bit으로, 포트 a[3:0], b[3:0]을 받아 덧셈을 수행하는 설계 모듈이다. 덧셈 결과의 하위 4비트는 s[3:0]에 연결되며, 상위 비트는 cout으로 연결된다.

```
adder_4bit.v 1.U X tb_adder_4bit.v 5.U X wavevcd simulation.json M X
sim > @ tb_adder_4bit.v > {} tb_adder_4bit > initial > {} for
3 module tb_adder_4bit;
19   initial begin
22     for(n=0; n<256; n=n+1) begin
29       checked = checked + 1;
30     end
31
32     if(checked != 256) $fatal(1, "Incomplete test");
33     $display("LAB1_PASS adder_4bit cases=%0d", checked);
34     $finish;
35   end
36
37   initial begin #100000; $fatal(1, "Watchdog"); end
38
39 endmodule
```

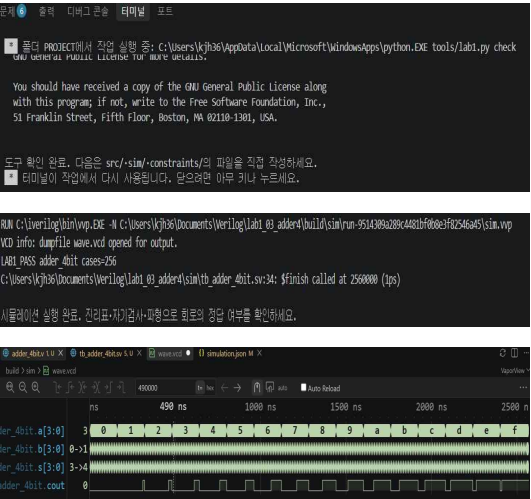
TB는 설계된 가산기를 자동으로 검증하기 위한 모듈이다. DUT로 adder_4bit를 연결하고, 가능한 모든 입력 조합을 발생시켜 실제 출력과 기댓값을 비교한다. 또한 VCD 파일을 생성하여 파형 분석을 가능하게 한다.

· 입력 조합, 기댓값, 자동 비교, 종료 조건

해당 TB는 4비트 입력 a와 b의 모든 조합을 검사한다. 가능한 입력 조합은 $16 \times 16 = 256$ 이다. 각 입력 조합은 10ns 동안 유지되며 다음 입력으로 변경된다. 기댓값은 $a+b$ 의 결과를 5비트 값으로 계산하여 생성한다. 계산된 기댓값의 상위 비트와 하위 4비트를 이용해 cout과 s를 동시에 검사한다. 실제 출력과 기댓값을 비교할 때는 '!=' 연산을 사용하여 X와 Z 상태까지 검출한다. 만약 출력이 기댓값과 다르다면 '\$fatal'을 이용하여 즉시 시뮬레이션을 종료

하고 오류를 보고한다. 256개의 입력 조합 검사가 종료되면 'LAB1_PASS adder_4bit cases=256'라는 메시지를 출력하고 종료한다. 전체 시뮬레이션 종료 시각은 2560ns이며 watchdog 기능을 사용하여 테스트가 무한히 수행되는 상황을 방지한다.

· PASS Log와 Wave



· 시간 구간별 값 해석 & 예상값 비교

VCD 파형을 확인한 결과, TB는 입력 a[3:0], b[3:0]의 모든 조합을 10ns 간격으로 인가하며 가산기의 동작을 검증하였다. 파형에서 a는 0부터 15까지 순차적으로 증가하고, b는 각 a 값에 대해 0부터 15까지 반복적으로 변화함을 확인하였다.

시간 (ns)	a	b	예상	실제	비교
5	0	0	cout=0, s=0	cout=0, s=0	同
15	0	1	cout=0, s=1	cout=0, s=1	同
35	0	3	cout=0, s=3	cout=0, s=3	同
315	1	15	cout=1, s=0	cout=1, s=0	同
1285	8	0	cout=0, s=8	cout=0, s=8	同
2555	15	15	cout=1, s=14	cout=1, s=14	同

· 오류, 수정, 재실행 및 보드에서 확인할 입출력

RTL의 덧셈 연산자를 뺄셈 연산자로 변경하면 a=0, b=1에 대하여 cout=1, s=1111이 되므로 TB가 불일치를 검출하고 \$fatal로 실행을 중단한다.

```
ERROR: 중도 코드 4: C:\Users\kjh36\Documents\Verilog\lab1_03_adder4\src -i C:\Users\kjh36\Documents\Verilog\lab1_03_adder4\src -s tb_adder_4bit -o C:\Users\kjh36\Documents\Verilog\lab1_03_adder4\build\simrun -c10f33c7b1a05e93ecdf63077b3a3\compile.log
I give up.
```

다시 '-'연산자를 '+'로 바꿔주면 정상적으로 PASS함을 알 수 있다. 보드 실험에서는 다음의 입출력을 확인한다.

입력	
DIP1~4	a[3:0]
DIP5~8	b[3:0]

출력	
LED 1	cout
LED 2 ~ LED 5	s[3:0]

4. 4비트 감산기

· 목적

Verilog HDL을 이용하여 4-bit Subtractor를 설계하고 시뮬레이션을 통해 검증한다. 또한, 감산 결과와 borrow 신호의 생성 원리를 이해하고 VS code에서 회로를 검증하는 방법을 학습한다.

· 포트 및 비트 폭

포트	방향	폭	설명
a	In	[3:0]	피감수(Minuend)
b	In	[3:0]	감수(Subtrahend)
d	Out	[3:0]	감산 결과의 하위 4비트
bor	Out	1bit	Borrow 발생 여부

· 동작 원리

4-bit Subtractor는 두 입력 a, b의 차를 계산한다. 만약 a가 b보다 작으면 음수 결과가 발생하므로 borrow가 발생한다. 예를 들어 '0-1 = -1'을 4비트 2의 보수 형태로 표현하면 '1111'이 되고 borrow는 1이 된다.

· 예상 결과표

a	b	d	bor
0	0	0	0
0	1	15	1
1	1	0	0
15	0	15	0
8	0	8	0
15	15	0	0

· RTL & TB

```

src > sub_4bit.v > {} sub_4bit
1 module sub_4bit(
2     input wire [3:0] a, b,
3     output wire [3:0] d,
4     output wire bor
5 );
6
7     assign d = a - b;
8     assign bor = a < b; // Equal operands do NOT borrow.
9
10 endmodule

```

RTL 모듈명은 'sub_4bit'이다. RTL에서는 입력 a[3:0], b[3:0]을 받아 'd = a-b'를 계산하고, 동시에 'bor = (a<b)'를 출력한다. 따라서 출력 d는 감산 결과의 하위 4비트를 나타내고, bor는 borrow 여부를 나타낸다.

```

sim > tb_sub_4bit.v > {} tb_sub_4bit
1 `timescale 1ns/1ps
2
3 module tb_sub_4bit;
4
5     reg [3:0] a,b;
6     wire [3:0] d;
7     wire bor;
8
9     sub_4bit dut(a,b,d,bor);
10
11     integer n,checked=0;
12     reg [4:0] expected;
13
14     initial begin
15         $dumpfile("wave.vcd");
16         $dumpvars(0,tb_sub_4bit);
17
18         for(n=0;n<256;n=n+1) begin
19             {a,b}=n;
20             expected=((n/16)<(n%16)) ? 16 : 0 | (((n/16)-(n%16)) & 15);
21             #10;
22
23             if ((bor,d) != expected)
24                 $fatal(1,"FAIL sub_4bit vector=%0d expected=%0d actual=%h",n, expected, (bor,d));
25             checked=checked+1;
26         end
27
28         if(checked!=256) $fatal(1,"Incomplete test");
29         $display("LAB1_PASS sub_4bit cases=%0d",checked);
30         $finish;
31     end
32
33     initial begin #100000; $fatal(1,"Watchdog"); end
34
35 endmodule

```

TB 모듈명은 'tb_sub_4bit'이다. TB는 DUT로 sub_4bit을 인스턴스화하고 입력 조합을 자동으로 생성하여 회로를 검증한다. TB는 기댓값과 실제 출력값을 비교하여 잘못된 결과가 발생하면 '\$fatal'을 통해 중단한다. 또한, watchdog을 사용해 무한히 실행하는 것을 방지한다.

· 코드 커밋 & 도구 버전

```
PS C:\Users\kjh36\Documents\Verilog\lab1_04_subtractor4> git --version
git version 2.55.0.windows.5
PS C:\Users\kjh36\Documents\Verilog\lab1_04_subtractor4> python --version
Python 3.14.7
PS C:\Users\kjh36\Documents\Verilog\lab1_04_subtractor4> code --version
1.137.0
645f29cc3176500b4b5762ba887cf2a7f0ffdf2c
arm64
```

```
PS C:\Users\kjh36\Documents\Verilog\lab1_04_subtractor4> iverilog -V
Icarus Verilog version 14.0 (devel) ()
```

```
PS C:\Users\kjh36\Documents\Verilog\lab1_04_subtractor4> vvp -V
Icarus Verilog runtime version 14.0 (devel) ()
```

```
PS C:\Users\kjh36\Documents\Verilog\lab1_04_subtractor4> git branch --show-current
>>
main
PS C:\Users\kjh36\Documents\Verilog\lab1_04_subtractor4> git rev-parse HEAD
```

· 입력 조합

4비트 입력 a, b는 각각 0~15의 값을 가질 수 있다. 따라서 전체 입력 조합 수는 256이다.

· 기댓값 계산

TB 내부에서는 'expected = a-b'를 이용하여 기댓값을 계산한다. 상위 비트에는 borrow 값을 저장하고, 하위 4비트에는 계산 결과를 저장한다. 즉, 'expected[4] = (a<b), expected[3:0] = (a-b)'의 형태로 비교가 수행된다.

· 자동 비교

각 입력은 10ns 동안 유지된다. TB는 '실제 출력 = 기대 출력'인지 자동으로 검사하는데, 만약 'actual != expected'이면 '\$fatal'을 실행하여 시뮬레이션을 종료한다.

· 종료 조건

모든 검사를 통과하면 'LAB1_PASS sub_4bit cases=256'라는 메시지를 출력한 후 종료한다.

· PASS Log & Wave

```
문제 4 출력 디버그 콘솔 터미널 코드

PROJECT에서 작업 실행 중: C:\Users\kjh36\AppData\Local\Microsoft\WindowsApps\python.EXE tools\lab1.py check
GNU GENERAL PUBLIC LICENSE FOR MORE DETAILS.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/sim/constraints의 파일을 직접 작성하세요.
터미널이 작업에서 다시 사용됩니다. 닫으려면 아무 키나 누르세요.
```

```
문제 4 출력 디버그 콘솔 터미널 코드

RUN C:\iverilog\bin\iverilog.EXE -g2012 -hail -I C:\Users\kjh36\Documents\Verilog\lab1_04_subtractor4\src -I C:\Users\kjh36\Documents\Verilog\lab1_04_subtractor4\sim -s tb sub_4bit -o C:\Users\kjh36\Documents\Verilog\lab1_04_subtractor4\build\sim\run-h9f36d19d114864476f9f5ca27719\sim.vvp C:\Users\kjh36\Documents\Verilog\lab1_04_subtractor4\src\sub_4bit.v C:\Users\kjh36\Documents\Verilog\lab1_04_subtractor4\sim\tb_sub_4bit.vv ..
RUN C:\iverilog\bin\iverilog.EXE -W C:\Users\kjh36\Documents\Verilog\lab1_04_subtractor4\build\sim\run-h9f36d19d114864476f9f5ca27719\sim.vvp
V0 info: dumpfile vvp.wvd opened for output.
LAB1_PASS sub_4bit cases=256
C:\Users\kjh36\Documents\Verilog\lab1_04_subtractor4\sim\tb_sub_4bit.vv:30: $finish called at 2560000 (tps)

시뮬레이션 실행 완료. 전리프-기검사-파형으로 파형의 정합 여부를 확인하세요.
```



모두 d=0000, bor=0이 출력되는 것을 확인하였다. 특히, 교안에서 강조한 것처럼 0-1 → bor=1, 1-1 → bor=0으로 차이가 나타난다.

· 오류, 수정, 재실행 및 보드에서 확인할 입
출력 설명

borrow 계산식에 등호를 추가하면 'bor = (a<=b);'가 되므로 a=0, b=0일 때 bor=1이 되어버려 기댓값과 일치하지 않는다. 이때 TB가 이를 검출하여 '\$fatal'을 발생시켜 시뮬레이션을 중단한다.

[illegible]

해당 오류를 확인 후 다시 원래대로 수정하여 실행하면 정상적으로 성공 메시지가 출력된다. 그리고 보드에서 확인할 입출력은 다음과 같다.

보드 장치	연결 신호
DIP1 ~ 4	a[3:0]
DIP5 ~ 8	b[3:0]
LED 1	bor
LED 2 ~ 5	d[3:0]

5. 4비트 크기 비교기

- 목적

4비트 입력 a , b 크기를 비교하는 조합논리회로를 설계하고 시뮬레이션한다. 비교 결과를 3비트 출력으로 표현하며, $a > b$, $a = b$, $a < b$ 상태를 확인한다.

· 입출력 포트 및 비트 폭

입력	
a[3:0]	4bit 비교 입력 A
b[3:0]	4bit 비교 입력 B

출력	
o[2]	a>b
o[1]	a=b
o[0]	a<b

신호	비트 폭
a	4bit
b	4bit
o	3bit

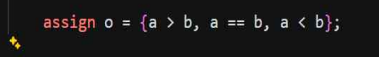
· 예상 결과표

a	b	의미	o[2:0]
0	0	a=b	010
0	1	a<b	001
1	0	a>b	100
8	0	a>b	100
15	15	a=b	010

· 동작 원리

비교기는 입력 a, b를 비교하여 다음과 같이 출력한다. ‘a>b이면 o=100’, ‘a=b이면 o=010’, ‘a<b이면 o=001’이다. 비교 결과는 동시에 여러 개가 참이 될 수 없으므로 항상 하나의 비트만 1이 된다.

- RTL & TB



```
1 module compare_4(  
2     input wire [3:0] a, b,  
3     output wire [2:0] o  
4 );  
5  
6     assign o = {a > b, a == b, a < b};  
7  
8 endmodule
```

RTL 모듈 'compare_4'는 4비트 입력 a[3:0], b[3:0]의 크기를 비교하는 조합논리 회로이다. 출력은 'o[2:0]'이며, 'o[2] =

$a > b$, $o[1] = a = b$, $o[0] = a < b$ 이다. 비교 결과에 따라 세 출력 중 하나만 1이 되도록 구성하였다.

```

1 `timescale 1ns/1ps
2
3 module tb_compare_4;
4
5     reg [3:0] a,b;
6     wire [2:0] o;
7
8     compare_4 dut(a, b, o);
9
10    integer n, checked=0;
11    reg [2:0] expected;
12
13    initial begin
14        $dumpfile("wave.vcd");
15        $dumpvars(0, tb_compare_4);
16
17        for(n=0; n<256; n=n+1) begin
18            {a,b}=n;
19            expected=((n/16)>(n%16)) ? 4 :
20                (((n/16)==(n%16)) ? 2 : 1);
21            #10;
22

```

```

23            if (o != expected)
24                $fatal(1,
25                    "FAIL compare_4 vector=%0d expected=%h actual=%h",
26                    n, expected, o);
27
28            checked=checked+1;
29        end
30        if(checked==256)
31            $fatal(1, "Incomplete test");
32        $display("LAB1_PASS compare_4 cases=%0d", checked);
33        $finish;
34    end
35    initial begin
36        #100000;
37        $fatal(1, "Watchdog");
38    end
39 endmodule

```

TB는 comparator의 기능을 자동으로 검증한다. DUT로 compare_4 모듈을 연결하고, 4비트 입력 a와 b의 모든 조합을 순차적으로 인가한다. 각 입력 조합마다 기댓값(100, 010, 001)을 계산한 뒤 실제 출력과 비교한다.

· 코드 커밋 & 도구 버전

```

PS C:\Users\kjh36\Documents\Verilog\lab1_05_comparator4> git --version
git version 2.55.0.windows.5
PS C:\Users\kjh36\Documents\Verilog\lab1_05_comparator4> python --version
Python 3.14.7
PS C:\Users\kjh36\Documents\Verilog\lab1_05_comparator4> code --version
1.137.0
645f29cc3176500b4b5762ba887cf2a7f0ffdf2c
arm64

```

```

PS C:\Users\kjh36\Documents\Verilog\lab1_05_comparator4> iverilog -V
Icarus Verilog version 14.0 (dev) ( )

```

```

PS C:\Users\kjh36\Documents\Verilog\lab1_05_comparator4> vvp -V
Icarus Verilog runtime version 14.0 (dev) ( )

```

```

PS C:\Users\kjh36\Documents\Verilog\lab1_05_comparator4> git branch --show-current
main
PS C:\Users\kjh36\Documents\Verilog\lab1_05_comparator4> git rev-parse HEAD
a248b802aa2dec8def8caaf5175f12fc005f8f5c

```

· 입력 조합

TB는 4비트 입력 두 개에 대해 모든 경우를 검사하기 때문에 총 256가지의 조합이 나올 수 있다.

· 기댓값 계산

조건	기대 출력
$a > b$	100
$a = b$	010
$a < b$	001

· 자동 비교

TB는 각 입력을 10ns 동안 인가한다. 매 입력에 대해 ‘실제 출력 == 기대 출력’을 비교한다. 출력이 다르면 ‘\$fatal’로 시뮬레이션을 종료한다.

· 종료 조건

모든 검사 통과 시 ‘LAB1_PASS compare_4 cases=256’을 출력하고 ‘\$finish’로 종료한다.

· PASS Log & Wave

```

문제 출력 디버그 콘솔 터미널 포트
[!] 플러그인 PROJECT에서 작업 실행 중: C:\Users\kjh36\AppData\Local\Microsoft\WindowsApps\python
GNU GENERAL PUBLIC LICENSE for more details.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/sim/constraints의 파일을 직접 작성하세요.
[!] 터미널이 작업에서 다시 사용됩니다. 닫으려면 아무 키나 누르세요.

```

```

문제 출력 디버그 콘솔 터미널 포트
RUN C:\Verilog\bin\iverilog.exe -g2012 -s1 C:\Users\kjh36\Documents\Verilog\lab1_05_comparator4\src -I C:\Users\kjh36\Documents\Verilog\lab1_05_comparator4\src -o tb_compare_4 -o C:\Users\kjh36\Documents\Verilog\lab1_05_comparator4\build\sim\run-20120517212401ba012c3d8dc
a248b802aa2dec8def8caaf5175f12fc005f8f5c\src\compare_4.v C:\Users\kjh36\Documents\Verilog\lab1_05_comparator4\src\compare_4.v C:\Users\kjh36\Documents\Verilog\lab1_05_comparator4\src\compare_4.v --
LAB1_PASS compare_4 cases=256
C:\Users\kjh36\Documents\Verilog\lab1_05_comparator4\src\compare_4.v:35: $finish called at 250000 (1ps)

시뮬레이션 실행 완료. 전리모-리-검사-결과-으로-최종-의-경우-여부를-확인-하세요.
wave C:\Users\kjh36\Documents\Verilog\lab1_05_comparator4\build\sim\wave.vcd
[!] 터미널이 작업에서 다시 사용됩니다. 닫으려면 아무 키나 누르세요.

```



· 시간 구간별 값 해석 & 예상값 비교

시간 (ns)	a[3:0]	b[3:0]	o[2:0]	결과	비교
0~10	0000	0000	010	2	同
10~20	0000	0001	001	1	同
20~30	0000	0010	001	1	同
160~170	0001	0000	100	4	同
170~180	0001	0001	010	2	同

모든 시간 구간에서 출력값이 예상한 비교 결과와 동일하게 나타났다. $a>b$ 인 경우 100(4), $a=b$ 인 경우 010(2), $a<b$ 인 경우 001(1)이 출력되었으며, 세 출력 비트 중 항상 하나만 활성화되는 형태의 comparator가 정상적으로 확인되었다. 또한, 시뮬레이션 결과 ‘LAB1_PASS compare_4 cases=256’이 출력되어 모든 입력 조합에 대해 올바르게 동작함을 검증하였다.

· 오류, 수정, 재실행 및 보드에서 확인할 입출력 설명

동일 비교 출력인 '(a == b)'를 '(a != b)'로 변경할 시 'a=0, b=0'에서 010이 아닌 000을 출력하므로 TB가 불일치를 검출하여 시뮬레이션을 중지시킨다.

[illegible]

해당 오류 코드를 원상태로 복구하고 다시 실행하면 정상적으로 파형을 확인할 수 있다. 그리고 보드에서 확인할 입출력은 다음과 같다.

스위치	기능
DIP 1 ~ 4	a[3:0]
DIP 5 ~ 8	b[3:0]

LED	의미
LED 1	$a > b$
LED 2	$a = b$
LED 3	$a < b$

6. 4:1 MUX

- 목적

4x1 MUX의 동작 원리를 이해한다. 선택 신호에 따라 여러 입력 중 하나를 출력으로 전달하는 조합논리회로를 Verilog HDL로 설계하고 시뮬레이션하여 검증한다.

· 입출력 포트 및 비트 폭

포트	방향	폭	설명
i[3:0]	In	4bit	데이터 입력
s[1:0]	In	2bit	선택 신호
z	Out	1bit	출력

· 동작 원리

선택 신호 s 값에 따라 출력 z 는 다음과 같이 결정된다.

s[1:0]	z
00	i[3]
01	i[2]
10	i[1]
11	i[0]

즉, 'z = i[3-s]'와 동일한 기능을 수행한다.
선택선이 증가할수록 입력 비트 번호는 감소

하는 구조이다.

· 예상 결과표

i	s	z
1000	00	1
1000	01	0
1000	10	0
1000	11	0
0001	00	0
0001	01	0
0001	10	0
0001	11	1

· RTL & TB

```
1 module mux_4x1(
2     input wire [3:0] i,
3     input wire [1:0] s,
4     output wire z
5 );
6
7     assign z = i[3-s];
8
9 endmodule
```

RTL은 4비트 입력 버스 i[3:0]와 2비트 선택 신호 s[1:0]를 받아 선택된 입력을 출력 z로 전달하는 조합논리회로이다.

```
1 `timescale 1ns/1ps
2
3 module tb_mux_4x1;
4
5     reg [3:0] i;
6     reg [1:0] s;
7     wire z;
8
9     mux_4x1 dut(i,s,z);
10
11     integer n,checked=0;
12     reg [0:0] expected;
13
14     initial begin
15         $dumpfile("wave.vcd");
16         $dumpvars(0,tb_mux_4x1);
17
18         for(n=0;n<64;n=n+1) begin
19             {i,s}=n;
20             expected=((n/4)>>(3-(n%4))) & 1;
21             #10;
22         end
```

```
23         $fatal(1,
24             "FAIL mux_4x1 vector=%0d expected=%h actual=%h",
25             n, expected, z);
26
27         checked=checked+1;
28     end
29     if(checked!=64)
30         $fatal(1,"Incomplete test");
31
32     $display("LAB1_PASS mux_4x1 cases=%0d",checked);
33     $finish;
34 end
35 initial begin
36     #100000;
37     $fatal(1,"Watchdog");
38 end
39 endmodule
```

TB는 DUT인 'mux_4x1'을 인스턴스화한 후, 입력 I의 16가지 값, 선택선 s의 4가지 값을 모두 조합하여 총 64개의 조합을 검사한다. 각 입력은 10ns 동안 유지되며 기댓값과 실제 출력을 비교한다.

· 코드 커밋 & 도구 버전

```
PS C:\Users\kjh36\Documents\Verilog\lab1_06_mux4to1> git --version
git version 2.55.0.windows.5
PS C:\Users\kjh36\Documents\Verilog\lab1_06_mux4to1> python --version
Python 3.14.7
PS C:\Users\kjh36\Documents\Verilog\lab1_06_mux4to1> code --version
1.137.0
645f29cc3176500b4b5762ba887cf2a7f0ffdf2c
arm64
```

```
PS C:\Users\kjh36\Documents\Verilog\lab1_06_mux4to1> iverilog -V
Icarus Verilog version 14.0 (dev) ()
```

```
PS C:\Users\kjh36\Documents\Verilog\lab1_06_mux4to1> vvp -V
Icarus Verilog runtime version 14.0 (dev) ()
```

```
PS C:\Users\kjh36\Documents\Verilog> cd lab1_06_mux4to1
PS C:\Users\kjh36\Documents\Verilog\lab1_06_mux4to1> git branch --show-current
main
PS C:\Users\kjh36\Documents\Verilog\lab1_06_mux4to1> git rev-parse HEAD
a248b802aa2dec8def8caaf5175f12fc005f8f5c
```

· 입력 조합

TB는 'i = 0000 ~ 1111' (16개), 's = 00 ~ 11' (4개)의 모든 조합을 검사한다. 즉, $16 \times 4 = 64$ 가지이다.

· 기댓값

기댓값은 'expected = i[3-s]'의 규칙을 따른다. 예를 들어 'i = 1000, s = 00'이면 z = 1'이다.

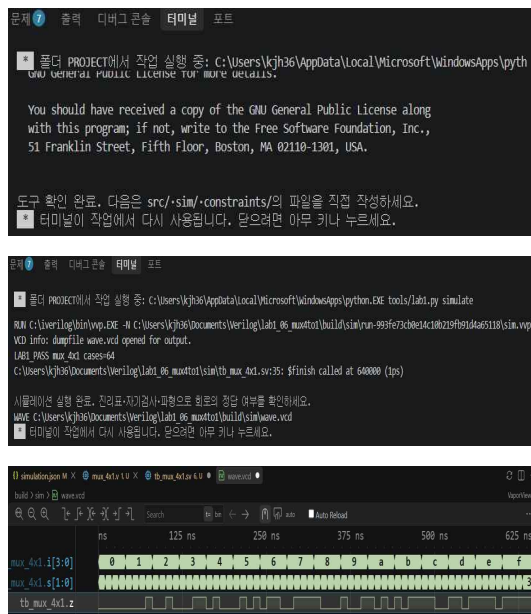
· 자동 비교

TB는 실제 출력 z와 기댓값을 비교한다. 기댓값과 실제 출력이 다르면 '\$fatal' 때문에 시뮬레이션이 중단되고 오류가 발생한다.

· 종료 조건

모든 검사를 통과하면 'LAB1_PASS mux_4x1 cases=64'를 출력하고 종료한다.

· PASS Log & Wave



· 시간 구간별 값 해석 및 예상값 비교

시뮬레이션에서 입력 i[3:0]은 10ns 간격으로 0~15(16진수 0~f)까지 증가하고, 각 입력 값에 대해 선택선 s[1:0]은 00 → 01 → 10 → 11 순으로 변화하였다. 4×1 MUX는 선택선 값에 따라 i[3], i[2], i[1], i[0] 중 하나를 출력으로 전달한다.

구간 1 (0~40ns)

i[3:0]	s[1:0]	Bit	예상 z	파형 z
0000	00~11	0,0,0,0	0	0

입력이 모두 0이므로 선택선과 관계없이 출력은 항상 0이다. 파형에서도 z가 계속

Low 상태를 유지하여 예상과 일치하였다.

구간 2 (40~80ns)

i[3:0]	s[1:0]	Bit	예상 z
0001	00	i[3]=0	0
0001	01	i[2]=0	0
0001	10	i[1]=0	0
0001	11	i[0]=1	1

입력이 0001일 때는 's=11'에서만 i[0]이 선택되어 출력이 1이 된다. 파형에서도 마지막 구간에서만 z가 High가 되어 예상과 일치하였다.

구간 3 (320~360ns)

i[3:0]	s[1:0]	Bit	예상 z
1000	00	i[3]=1	1
1000	01	i[2]=0	0
1000	10	i[1]=0	0
1000	11	i[0]=0	0

i=1000 상태에서 선택선을 변화시키면 i[3]만 1이고 나머지 비트는 모두 0이므로 s=00일 때만 출력이 1이 된다. 실제 파형에서도 첫 구간만 High가 나타나고 이후에는 모두 Low가 나타나 예상값과 정확히 일치하였다.

구간 4 (600~640ns)

i[3:0]	s[1:0]	Bit	예상 z
1111	00	i[3]=1	1
1111	01	i[2]=1	1
1111	10	i[1]=1	1
1111	11	i[0]=1	1

입력이 모두 1인 경우에는 어떤 비트를 선택하더라도 출력이 1이 된다. 파형에서도 z가 계속 High 상태를 유지하여 예상 결과와 일치하였다.

- 오류, 수정, 재실행 및 보드에서 확인할 입출력

‘i[3-s]’를 ‘i[s]’로 바꾸게 된다면 입력이 ‘i = 0001, s = 00’일 때, 출력이 ‘0(i[3])’이 아닌 ‘1(i[0])’이 나오게 되므로 TB가 이를 검출하여 PASS가 아닌 FAIL을 발생시킨다.

```

1 module mux_4to1
2   input wire [3:0] i,
3   input wire [1:0] s,
4   output wire z
5 );
6   assign z = i[s];
7
8
9 endmodule
  
```

‘i[s]’ 부분을 다시 원래대로 수정 후 재실행하게 되면 정상적인 PASS가 나옴을 확인할 수 있다. 보드에서 확인할 입출력은 다음과 같다.

입력	RTL 포트
KEY 1	i[3]
KEY 2	i[2]
KEY 3	i[1]
KEY 4	i[0]
DIP 1	s[1]
DIP 2	s[0]

출력	RTL 포트
LED 1	z

출력 중 하나로 전달하는 회로를 Verilog HDL로 설계하고 시뮬레이션한다.

- 포트 및 비트 폭

포트	방향	비트 폭	설명
i	In	1bit	입력
s[2:0]	In	3bit	select
o[7:0]	Out	8bit	출력

- 동작 원리

DEMUX는 하나의 입력 신호를 여러 출력 중 하나로 전달하는 회로이다.

s=000 → o[7] 선택

s=001 → o[6] 선택

s=010 → o[5] 선택

s=011 → o[4] 선택

s=100 → o[3] 선택

s=101 → o[2] 선택

s=110 → o[1] 선택

s=111 → o[0] 선택

또한, 입력 i=0이면 선택선 값과 관계없이 모든 출력이 0이 된다.

- 예상 결과표

i = 0

7. 1:8 DE-MUX

- 목적

1x8 DE-MUX의 동작 원리를 이해한다. 하나의 입력 신호를 선택선 값에 따라 8개의

s[2:0]	o[7:0]
000	00000000
001	00000000
010	00000000
011	00000000
100	00000000
101	00000000
110	00000000
111	00000000

i = 1

s[2:0]	o[7:0]
000	10000000
001	01000000
010	00100000
011	00010000
100	00001000
101	00000100
110	00000010
111	00000001

· RTL & TB

```

1 module demux_1x8(
2     input wire i,
3     input wire [2:0] s,
4     output wire [7:0] o
5 );
6
7 // Preserve the original lab ordering: 000 -> o[7].
8 assign o = i ? (8'b10000000 >> s) : 8'b0;
9
10 endmodule

```

입력 i와 선택선 s[2:0]을 받아 8비트 출력 o[7:0]을 생성한다. 선택선 값에 따라 입력 i가 하나의 출력 비트로 전달되며 나머지 비트는 0이 된다. 구현은 입력 i를 기준으로 10000000 값을 선택선 값만큼 오른쪽 이동

시키는 방식으로 작성된다.

```

1 `timescale 1ns/1ps
2 module tb_demux_1x8;
3
4     reg i;
5     reg [2:0] s;
6     wire [7:0] o;
7
8     demux_1x8 dut(i, s, o);
9
10    integer n, checked=0;
11    reg [7:0] expected;
12
13    initial begin
14        $dumpfile("wave.vcd");
15        $dumpvars(0, tb_demux_1x8);
16
17        for(n=0; n<16; n=n+1) begin
18            {i, s}=n;
19            expected=(n>=8) ? (128 >> (n%8)) : 0;
20            #10;
21

```

```

22            if (o != expected)
23                $fatal(1,
24                    "FAIL demux_1x8 vector=%0d expected=%h actual=%h",
25                    n, expected, o);
26
27            checked=checked+1;
28        end
29        if(checked!=16)
30            $fatal(1, "Incomplete test");
31
32        $display("LAB1_PASS demux_1x8 cases=%0d", checked);
33        $finish;
34    end
35    initial begin
36        #1000000;
37        $fatal(1, "Watchdog");
38    end
39 endmodule

```

TB는 DUT(Demux 회로) 인스턴스 생성, 16가지 입력 조합 자동 생성, 기댓값 계산, 실제 출력과 자동 비교, VCD 파형 파일 생성, PASS/FAIL 출력, watchdog를 이용한 비정상 종료 감지의 기능을 수행한다.

· 코드 커밋 & 도구 버전

```

PS C:\Users\kjh36\Documents\Verilog\lab1_07_demuxito8> git --version
git version 2.55.0.windows.5
PS C:\Users\kjh36\Documents\Verilog\lab1_07_demuxito8> python --version
Python 3.14.7
PS C:\Users\kjh36\Documents\Verilog\lab1_07_demuxito8> code --version
1.137.0
645f29cc3176500b4b5762ba887cf2a7f0ffdf2c
arm64

```

```

PS C:\Users\kjh36\Documents\Verilog\lab1_07_demuxito8> iverilog -V
Icarus Verilog version 14.0 (devel) ()

```

```

PS C:\Users\kjh36\Documents\Verilog\lab1_07_demuxito8> vvp -V
Icarus Verilog runtime version 14.0 (devel) ()

```

```
PS C:\Users\kjh36\Documents\Verilog\lab1_07_demux1to8> git branch --show-current
main
PS C:\Users\kjh36\Documents\Verilog\lab1_07_demux1to8> git rev-parse HEAD
>>
a248b802aa2dec8def8caaf5175f12fc005f8f5c
```

· 입력 조합

i	s[2:0]
0	000~111
1	000~111

· 기댓값

i=0 → 출력은 항상 00000000

i=1 → 선택선에 대응하는 출력 한 비트만 1

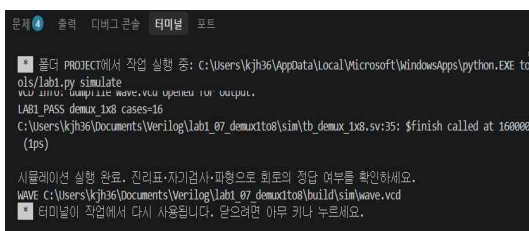
· 자동 비교

TB에서 계산한 기댓값과 DUT 출력 o를 비교한다. 값이 다르면 즉시 FAIL 처리된다.

· 종료 조건

모든 테스트 통과 시 'LAB1_PASS demux_1x8 cases=16' 출력 후 종료한다.

· PASS Log & Wave



· 시간 구간별 값 해석 및 예상값 비교

시간 구간(ns)	i	s[2:0]	실제 출력(Hex)	실제 출력(Binary)	예상 출력	비교
0~10	0	000	00	00000000	00000000	일치
10~20	0	001	00	00000000	00000000	일치
20~30	0	010	00	00000000	00000000	일치
30~40	0	011	00	00000000	00000000	일치
40~50	0	100	00	00000000	00000000	일치
50~60	0	101	00	00000000	00000000	일치
60~70	0	110	00	00000000	00000000	일치
70~80	0	111	00	00000000	00000000	일치
80~90	1	000	80	10000000	10000000	일치
90~100	1	001	40	01000000	01000000	일치
100~110	1	010	20	00100000	00100000	일치
110~120	1	011	10	00010000	00010000	일치
120~130	1	100	08	00001000	00001000	일치
130~140	1	101	04	00000100	00000100	일치
140~150	1	110	02	00000010	00000010	일치
150~160	1	111	01	00000001	00000001	일치

0~80ns 구간

입력 i=0 상태이다. 선택선 s가 000부터 111까지 변화하지만 출력은 항상 00000000이다.

80~90ns 구간

i=1, s=000, 출력은 10000000(0x80)이 되며 o[7]만 1이므로 예상값과 동일하다.

90~150ns 구간

선택선이 증가함에 따라 출력의 1이 o[6] → o[1] 방향으로 이동한다. 파형에서 40 → 20 → 10 → 08 → 04 → 02로 표시되며 예상값과 동일하다.

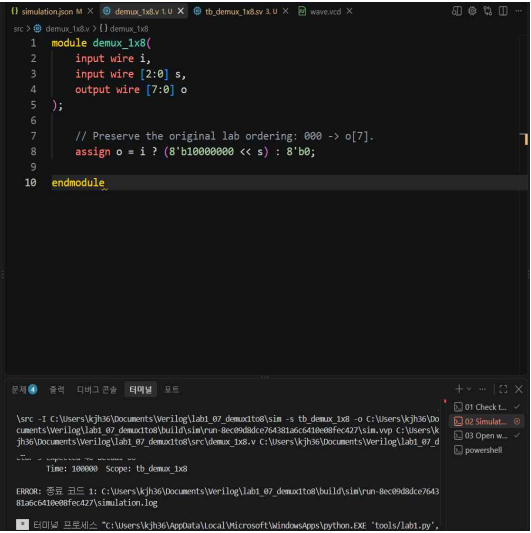
150~160ns 구간

i=1, s=111, 출력은 00000001(0x01)이 되어 o[0]만 1이 된다. 이는 DE-MUX의 마지막 출력이 선택된 경우이며 예상값과 일치한다.

· 오류, 수정, 재실행 및 보드에서 확인할 입출력

정상 동작 코드의 오른쪽 시프트 연산을 원

쪽 시프트 연산으로 변경한다. '128 >> s'
→ '128 << s'. 이렇게 변경하면 'i = 1, s
= 001'일 때 '01000000'이 아닌 '00000000'
이 나오게 되어 TB에서 FAIL이 뜨게 된다.



```
1 module demux_1x8(  
2     input wire i,  
3     input wire [2:0] s,  
4     output wire [7:0] o  
5 );  
6  
7 // Preserve the original lab ordering; 000 -> o[7].  
8 assign o = i ? (8'b10000000 << s) : 8'b0;  
9  
10 endmodule
```

Simulation results for tb_demux_1x8:

```
Time: 100000 Scope: tb_demux_1x8  
ERROR: 종료 코드 1: C:\Users\kjh36\Documents\Verilog\lab1_07_demux\src\demux_1x8.v: C:\Users\kjh36\Documents\Verilog\lab1_07_d  
81a6c6d10e08fec427\simulation.log
```

해당 부분을 다시 오른쪽 시프트 연산자로
변경한 뒤 재실행하면 정상적으로 PASS가
나옴을 알 수 있다. 그리고 보드에서 확인할
입출력은 다음과 같다.

입력

보드 장치	신호
KEY1	i
DIP1	s[2]
DIP2	s[1]
DIP3	s[0]

출력

보드 LED	신호
LED1	o[7]
LED2	o[6]
LED3	o[5]
LED4	o[4]
LED5	o[3]
LED6	o[2]
LED7	o[1]
LED8	o[0]

8. 8:3 인코더

· 목적

8x3 Encoder를 설계하고 Verilog HDL로
구현한다. 입력선 8개 중 단 하나의 입력만
1인 경우 해당 입력의 위치를 3비트 이진수
로 변환하여 출력하는 동작을 확인한다. 또
한, 잘못된 입력(모든 입력이 0 or 여러 입
력이 동시에 1)에 대해 출력이 000이 되는
것을 검증한다.

· 포트 및 비트 폭

신호	방향	폭	설명
i[7:0]	In	8bit	입력
a[2:0]	Out	3bit	출력

· 동작 원리

본 회로는 우선순위 인코더가 아닌 일반 인
코더이다.

- i[7]만 1 → a=000
- i[6]만 1 → a=001
- i[5]만 1 → a=010
- i[4]만 1 → a=011
- i[3]만 1 → a=100
- i[2]만 1 → a=101
- i[1]만 1 → a=110
- i[0]만 1 → a=111

입력이 없거나 여러 비트가 동시에 1이면
유효한 one-hot 입력이 아니므로 출력은
000이 된다.

· 예상 결과표

입력 i[7:0]	출력 a[2:0]
00000000	000
00000001	111
00000010	110
00000100	101
00001000	100
00010000	011
00100000	010
01000000	001
10000000	000
00000011	000
11111111	000

· RTL & TB

```

1 module encoder8x3(
2     input wire [7:0] i,
3     output reg [2:0] a
4 );
5
6 always @* begin
7     case (i)
8         8'h80: a = 0;
9         8'h40: a = 1;
10        8'h20: a = 2;
11        8'h10: a = 3;
12        8'h08: a = 4;
13        8'h04: a = 5;
14        8'h02: a = 6;
15        8'h01: a = 7;
16        default: a = 0;
17    endcase
18 end
19
20 endmodule

```

입력 i[7:0]에 대해 case 문을 사용하여 각 One-Hot 입력을 대응하는 3비트 출력값으로 변환한다. 조건에 해당하지 않을 때는 default 문을 사용하여 출력을 000으로 설정한다.

```

1 `timescale 1ns/1ps
2 module tb_encoder8x3;
3
4     reg [7:0] i;
5     wire [2:0] a;
6     integer k;
7     encoder8x3 dut(i, a);
8     integer n, checked=0;
9     reg [2:0] expected;
10
11     initial begin
12         $dumpfile("wave.vcd");
13         $dumpvars(0, tb_encoder8x3);
14         for(n=0; n<256; n=n+1) begin
15             i=n;
16             expected=0;
17             for(k=0; k<8; k=k+1)
18                 if(n==(128>>k))
19                     expected=k;
20             #10;
21         end
22     end
23
24     if (a != expected)
25         $fatal(1,
26             "FAIL encoder8x3 vector=%0d expected=%0h actual=%0h",
27             n, expected, a);
28     checked=checked+1;
29 end
30
31 if(checked!=256)
32     $fatal(1, "Incomplete test");
33
34 $display("LAB1_PASS encoder8x3 cases=%0d", checked);
35 $finish;
36
37 initial begin
38     #100000;
39     $fatal(1, "Watchdog");
40 end
41 endmodule

```

TB는 0~255까지의 모든 입력 조합 생성, 기댓값 계산, DUT 출력과 기댓값 비교, 값이 다르면 '\$fatal'로 즉시 종료, 모든 검사가 통과하면 PASS 출력, VCD 파형 파일 생성, 'Watchdog'으로 무한 시뮬레이션 방지, 총 256개의 입력 조합을 자동 검증하는 기능을 수행한다.

· 입력 조합

TB는 8비트 입력의 모든 경우인 256개 조합을 순차적으로 입력한다.

· 기댓값

One-hot인 경우 : 해당 번호 출력
0개 or 2개 이상 비트가 1 : 000 출력

· 자동 비교

TB에서 DUT 출력과 기댓값을 자동 비교한

다.

· 종료 조건

모든 테스트를 통과하면 'LAB1_PASS encdoer8x3 cases=256'을 출력한 뒤 종료한다.

· 코드 커밋 & 도구 버전

```
PS C:\Users\Kjh36\Documents\Verilog\lab1_08_encoder8to3> git --version
git version 2.55.0.windows.5
PS C:\Users\Kjh36\Documents\Verilog\lab1_08_encoder8to3> python --version
Python 3.14.7
PS C:\Users\Kjh36\Documents\Verilog\lab1_08_encoder8to3> code --version
1.137.0
645f29cc317650b4b5762ba887cf2a7f0ffdf2c
arm64
```

```
PS C:\Users\Kjh36\Documents\Verilog\lab1_08_encoder8to3> iverilog -V
Icarus Verilog version 14.0 (devel) ()
```

```
PS C:\Users\Kjh36\Documents\Verilog\lab1_08_encoder8to3> vvp -V
Icarus Verilog runtime version 14.0 (devel) ()
```

```
PS C:\Users\Kjh36\Documents\Verilog\lab1_08_encoder8to3> git branch --show-current
main
PS C:\Users\Kjh36\Documents\Verilog\lab1_08_encoder8to3> git rev-parse HEAD
a248b02aa2dec8def8caaf5175f12fc005f8f5c
```

· PASS Log & Wave

문제 출력 디버그 콘솔 터미널 포트

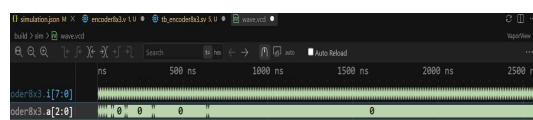
❏ 물디 PROJECT에서 작업 실행 중: C:\Users\Kjh36\AppData\Local\Microsoft\WindowsApps\python.exe
GNU GENERAL PUBLIC LICENSE FOR MORE DETAILS.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/-constraints/의 파일을 직접 작성하세요.
❏ 터미널이 작업에서 다시 사용됩니다. 알으려면 아무 키나 누르세요.

문제 출력 디버그 콘솔 터미널 포트

```
Run C:\Verilog\bin\iverilog -g2012 -s1 C:\Users\Kjh36\Documents\Verilog\lab1_08_encoder8to3\src -I C:\Users\Kjh36\Documents\Verilog\lab1_08_encoder8to3\src -o C:\Users\Kjh36\Documents\Verilog\lab1_08_encoder8to3\bin\lab1_08_encoder8to3.vvp C:\Users\Kjh36\Documents\Verilog\lab1_08_encoder8to3\src\encoder8to3.v C:\Users\Kjh36\Documents\Verilog\lab1_08_encoder8to3\src\lab1_08_encoder8to3.v ...
Run C:\Verilog\bin\vvp -x64 -I C:\Users\Kjh36\Documents\Verilog\lab1_08_encoder8to3\bin\src -r6472a6d73c5d6f201c087a08\sim.vvp -vcd -vcdfile dumpfile named opened for output.
LAB1_PASS_encoder8to3 cases=256
C:\Users\Kjh36\Documents\Verilog\lab1_08_encoder8to3\bin\lab1_08_encoder8to3.v:48: $finish called at 256000 (tps)
시뮬레이션 실행 완료. 결과표, 지그(감사)파일으로 원본의 경우 결과를 확인하세요.
```



· 시간 구간별 값 해석 및 예상값 비교

시뮬레이션 파형에서 입력 i[7:0]와 출력 a[2:0]를 관찰하였다. TB는 0~255의 모든 입력 조합을 10ns 간격으로 인가하며 총 2560ns 동안 동작한다.

시간(ns)	입력 i[7:0]	예상 출력 a[2:0]	결과 해석
5	00000000	000	모든 입력이 0이므로 무효 입력이다. 출력 000이 예상과 일치한다.
15	00000001	111	i[0]만 1인 one-hot 입력이다. 인코더는 입력 번호 7을 출력하며 예상값과 일치한다.
25	00000010	110	i[1]만 1인 one-hot 입력이다. 출력은 110이며 예상값과 일치한다.
35	00000011	000	두 비트가 동시에 1이므로 무효 입력이다. 출력 000이 예상과 일치한다.
1285	10000000	000	i[7]만 1인 one-hot 입력이다. 입력 번호 0에 대응하여 000이 출력된다.
2555	11111111	000	8개 입력이 모두 1인 다중 입력 상태이다. 출력 000이 예상과 일치한다.

시뮬레이션 결과를 확인한 결과, one-hot 입력에서는 해당 입력 위치에 대응하는 3비트 값이 정상적으로 출력되었고, 입력이 없거나 여러 비트가 동시에 1인 경우에는 모두 000이 출력되었다. 특히 i=00000001일 때 출력이 111로 나타나고, i=00000011일 때 출력이 000으로 나타나는 것을 통해 본 회로가 우선순위 인코더가 아닌 일반 8x3 Encoder임을 확인할 수 있었다. 또한, 전체 256개 입력 조합에 대해 PASS 로그가 출력되므로 RTL과 TB가 예상된 기능을 정상적으로 수행한 것으로 판단된다.

· 오류, 수정, 재실행 및 보드에서 확인할 입력 출력

RTL에서 입력 '00000001'에 대응하는 출력 상수를 7에서 6으로 변경한다. 이렇게 되면 입력이 'i = 00000001'일 때 기댓값은 111이지만 실제값은 110이 나오게 되어 TB가 FAIL을 검출한다.

```

1 module encoder8x3(
2     input wire [7:0] i,
3     output reg [2:0] a
4 );
5
6 always @* begin
7     case (i)
8         8'h00: a = 0;
9         8'h01: a = 1;
10        8'h02: a = 2;
11        8'h03: a = 3;
12        8'h04: a = 4;
13        8'h05: a = 5;
14        8'h06: a = 6;
15        8'h07: a = 7;
16        default: a = 0;
17    endcase
18 end
19
20 endmodule

```

해당 부분을 다시 7로 변경한 뒤 재실행하면 오류 없이 PASS가 나오게 된다. 그리고 보드에서 확인할 입출력은 다음과 같다.

입력(KEY)

보드 스위치	신호
KEY1~KEY8	i[7:0]

출력(LED)

보드 LED	신호
LED1~LED3	a[2:0]

9. 3:8 인코더

· 목적

3x8 Decoder의 동작 원리를 이해하고 Verilog HDL로 구현한다. 또한, 시뮬레이션을 통해 입력 조합에 따른 출력 변화를 확인하고 검증한다.

· 포트 및 비트 폭

포트	방향	비트 폭	설명
a	Input	1bit	입력 A
b	Input	1bit	입력 B
c	Input	1bit	입력 C
o[7:0]	Output	8bit	디코더 출력

· 동작 원리

Decoder는 입력된 3비트 값을 8개의 출력 중 하나로 변환하는 조합논리회로이다. 입력 값 a, b, c를 하나의 이진수로 해석하여 해당 번호의 출력 비트만 1로 만들고 나머지는 모두 0으로 만든다.

· 예상 결과표

시간(ns)	a	b	c	abc(2진)	예상 출력 o[7:0]
0~10	0	0	0	0	00000001
10~20	0	0	1	1	00000010
20~30	0	1	0	2	00000100
30~40	0	1	1	3	00001000
40~50	1	0	0	4	00010000
50~60	1	0	1	5	00100000
60~70	1	1	0	6	01000000
70~80	1	1	1	7	10000000

· RTL & TB

```

1 module decoder3x8(
2     input wire a,
3     input wire b,
4     input wire c,
5     output wire [7:0] o
6 );
7
8     assign o = 8'b00000001 << {a,b,c};
9
10 endmodule

```

RTL은 3비트 입력 a,b,c를 하나의 3비트 숫자로 결합한 뒤, 8비트 값 00000001을 해당 값만큼 왼쪽으로 이동시켜 출력한다.

```

1 `timescale 1ns/1ps
2
3 module tb_decoder3x8;
4
5     reg a,b,c;
6     wire [7:0] o;
7
8     decoder3x8 dut(a,b,c,o);
9
10    integer n,checked=0;
11    reg [7:0] expected;
12
13    initial begin
14        $dumpfile("wave.vcd");
15        $dumpvars(0,tb_decoder3x8);
16
17        for(n=0;n<8;n=n+1) begin
18            {a,b,c}=n;
19            expected=1 << n;
20            #10;
21        end
22    end

```

```

22         if (o != expected)
23             $fatal(1,
24                 "FAIL decoder3x8 vector=%0d expected=%h actual=%h",
25                 n, expected, o);
26         checked=checked+1;
27     end
28     if (checked!=8)
29         $fatal(1,"Incomplete test");
30
31     $display("LAB1_PASS decoder3x8 cases=%0d",checked);
32     $finish;
33 end
34 initial begin
35     #100000;
36     $fatal(1,"Watchdog");
37 end
38 endmodule_

```

TB는 a, b, c를 000~111까지 순차 입력, 각 입력을 10ns 동안 유지, 기댓값 계산, 실제 출력(o)과 비교, 불일치 시 '\$fatal' 발생, 8개 조합 모두 통과하면 PASS 출력, 80ns에서 정상 종료, wave.vcd 파형 생성, Watchdog로 비정상 실행 방지를 수행한다.

· 코드 커밋 & 도구 버전

```

PS C:\Users\kjh36\Documents\Verilog\lab1_09_decoder3to8> git --version
>> python --version
>> code --version
git version 2.55.0.windows.5
Python 3.14.7
1.137.0
645f29cc3176500b4b5762ba887cf2a7f0ffdf2c
arm64

```

```

PS C:\Users\kjh36\Documents\Verilog\lab1_09_decoder3to8> iverilog -V
Icarus Verilog version 14.0 (dev) ()

```

```

PS C:\Users\kjh36\Documents\Verilog\lab1_09_decoder3to8> vvp -V
Icarus Verilog runtime version 14.0 (dev) ()

```

```

PS C:\Users\kjh36\Documents\Verilog\lab1_09_decoder3to8> git branch --show-current
>> git rev-parse HEAD
main
a248b802aa2dec8def8caaf5175f12fc005f8f5c

```

· 입력 조합

TB는 다음 8개 입력을 모두 검사한다.

순서	abc
1	000
2	001
3	010
4	011
5	100
6	101
7	110
8	111

· 기댓값 계산

입력값과 같은 번호의 출력 비트만 1이 된다.

· 자동 비교

TB는 실제 출력과 기댓값을 비교하여 오류 발생 시 '\$fatal'로 시뮬레이션을 종료한다.

· 종료 조건

모든 테스트를 통과 시 'LAB1_PASS decoder3x8 cases=8'을 출력 후 종료한다.

· PASS Log & Wave

문제 4 출력 디버그 콘솔 터미널 포트

폴더 PROJECT에서 작업 실행 중: C:\Users\kjh36\AppData\Local\Microsoft\WindowsApps\python.GNU GENERAL PUBLIC LICENSE FOR more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/~sim/~constraints/의 파일을 직접 작성하세요.

터미널이 작업에서 다시 사용됩니다. 닫으려면 아무 키나 누르세요.

문제 4 출력 디버그 콘솔 터미널 포트

폴더 PROJECT에서 작업 실행 중: C:\Users\kjh36\AppData\Local\Microsoft\WindowsApps\python.EXE tools\lab1.py simulate

RUN C:\Verilog\bin\vvp.EXE -W C:\Users\kjh36\Documents\Verilog\lab1_09_decoder3to8\build\sim\run-8c64a1895c47408f1d838863f04c\sim.vvp

VD info: dumpfile wave.vcd opened for output.

LAB1_PASS decoder3x8 cases=8

C:\Users\kjh36\Documents\Verilog\lab1_09_decoder3to8\sim\th_decoder3x8.sv:34: \$finish called at 80000 (tps)

시뮬레이션 실행 완료. 전이표-자극사-파형으로 원본의 검증 여부를 확인하세요.

WAVE C:\Users\kjh36\Documents\Verilog\lab1_09_decoder3to8\build\sim\wave.vcd

터미널이 작업에서 다시 사용됩니다. 닫으려면 아무 키나 누르세요.



· 시간 구간별 값 해석 및 예상값 비교

파형에서 a, b, c, o[7:0] 신호를 확인하였다. 각 입력 조합은 10ns 동안 유지되며, 출력은 입력값 a, b, c에 해당하는 번호의 비트만 1이 되는 형태로 나타났다.

시간 구간(ns)	입력 abc	예상 출력(2진수)	파형 출력(16진수)	비교 결과
0~10	000	00000001	01	일치
10~20	001	00000010	02	일치
20~30	010	00000100	04	일치
30~40	011	00001000	08	일치
40~50	100	00010000	10	일치
50~60	101	00100000	20	일치
60~70	110	01000000	40	일치
70~80	111	10000000	80	일치

0~10ns : 입력이 000이므로 출력 o[0]만 1이 되어 00000001(01)이 관측되었다.

10~20ns : 입력이 001로 변경되면서 출력 위치가 한 칸 이동하여 00000010(02)가 되었다.

20~30ns : 입력 010에 대해 o[2]만 1이 되어 00000100(04)가 출력되었다.

30~40ns : 입력 011에 대해 o[3]만 활성화되어 00001000(08)이 관측되었다.

40~50ns : 입력 100에 대해 o[4]만 1이 되어 00010000(10)이 출력되었다.

50~60ns : 입력 101에 대해 o[5]만 1인 00100000(20)이 출력되었다.

60~70ns : 입력 110에 대해 o[6]만 1인 01000000(40)이 출력되었다.

70~80ns : 입력 111에 대해 최상위 비트인 o[7]만 1이 되어 10000000(80)이 출력되었다.

· 오류, 수정, 재실행 및 보드에서 확인할 입출력

RTL에서 'assign o = 8'b00000001 << {a,b,c};'을 'assign o = 8'b00000001 >>

{a,b,c};'로 수정한다. 이렇게 되면 입력이 '001'일 때 정상 출력은 00000010이지만 실제로는 00000000이 출력되어 TB에서 FAIL이 발생한다.

```
1 simulation_run X
2 decoder3d.v X
3 decoder3d.v X
4 decoder3d.v X
5 assign
6
7 module decoder3d(
8     input wire a,
9     input wire b,
10    input wire c,
11    output wire [7:0] o
12);
13
14    assign o = 8'b00000001 >> {a,b,c};
15
16endmodule
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408

```

10. 7세그먼트 디코더

· 목적

4비트 입력값 bcd[3:0]을 받아 7-Segment LED 표시 코드로 변환하는 Seven Segment Decoder를 설계하고 검증한다.

· 포트 및 비트 폭

포트명	방향	비트 폭	설명
bcd	입력	[3:0]	표시할 16진수 값
seg_data	출력	[7:0]	a,b,c,d,e,f,g,dp 제어

· 동작 원리

Decoder는 입력 bcd 값에 따라 case문을 이용하여 미리 정의된 7-Segment 패턴을 출력한다.

· 예상 결과표

bcd	seg_data(hex)	표시
0	FC	0
1	60	1
2	DA	2
3	F2	3
4	66	4
5	B6	5
6	BE	6
7	E0	7
8	FE	8
9	F6	9
A	EE	A
B	3E	b
C	9C	C
D	7A	d
E	9E	E
F	8E	F

Seven Segment Decoder는 입력된 4비트 BCD 값을 조합논리로 해석하여 각 숫자 또는 문자(0~F)를 표시하기 위한 Segment 점등 패턴을 출력한다. 출력 seg_data[7:0]은 a,b,c,d,e,f,g,dp 순서로 구성되며, 각 비트가 1이면 해당 Segment가 켜진다. 따라서 입력값마다 사람이 읽을 수 있는 숫자 또는

문자 형태가 되도록 정해진 패턴을 출력해야 한다. 예를 들어 입력값이 8일 경우 a~g의 모든 Segment가 켜져야 하므로 11111110(FE)이 출력되며, 입력값이 F일 경우 a,e,f,g만 켜져 문자 F가 표시되므로 10001110(8E)이 출력된다.

· RTL & TB

```

1 module seg_decoder(
2     input wire [3:0] bcd,
3     output reg [7:0] seg_data
4 );
5
6 always @* begin
7     case (bcd)
8         0 : seg_data = 8'hfc;
9         1 : seg_data = 8'h60;
10        2 : seg_data = 8'hda;
11        3 : seg_data = 8'hf2;
12        4 : seg_data = 8'h66;
13        5 : seg_data = 8'hb6;
14        6 : seg_data = 8'hbe;
15        7 : seg_data = 8'he0;
16        8 : seg_data = 8'hfe;
17        9 : seg_data = 8'hf6;
18        10 : seg_data = 8'hee;
19        11 : seg_data = 8'h3e;
20        12 : seg_data = 8'h9c;
21        13 : seg_data = 8'h7a;
22        14 : seg_data = 8'h9e;
23        15 : seg_data = 8'h8e;
24        default : seg_data = 0;
25    endcase
26 end
27
28 endmodule
  
```

RTL은 입력 bcd[3:0]을 받고, always @(*) 조합논리로 설계한다. case문을 이용하여 16개의 입력(0~F)에 대한 seg_data 값을 출력하며 default 문으로 예외 상황을 처리한다.

```

1 `timescale 1ns/1ps
2
3 module tb_seg_decoder;
4
5     reg [3:0] bcd;
6     wire [7:0] seg_data;
7     reg [7:0] digits[0:15];
8
9     seg_decoder dut(bcd, seg_data, seg_data);
10
11     integer n, checked=0;
12     reg [7:0] expected;
13
14     initial begin
15         $dumpfile("wave.vcd");
16         $dumpvars(0, tb_seg_decoder);
17
18         digits[0]=8'hfc;
19         digits[1]=8'h60;
20         digits[2]=8'hda;
21         digits[3]=8'hf2;
22         digits[4]=8'h66;

```

```

23         digits[5]=8'hb6;
24         digits[6]=8'hbe;
25         digits[7]=8'he0;
26         digits[8]=8'hfe;
27         digits[9]=8'hf6;
28         digits[10]=8'hee;
29         digits[11]=8'h3e;
30         digits[12]=8'h9c;
31         digits[13]=8'h7a;
32         digits[14]=8'h9e;
33         digits[15]=8'h8e;
34
35         for(n=0;n<16;n=n+1) begin
36             bcd=n;
37             expected=digits[n];
38             #10;

```

```

39
40             if (seg_data != expected)
41                 $fatal(1,
42                     "FAIL seg_decoder vector=%0d expected=%0h actual=%0h",
43                     n, expected, seg_data);
44
45             checked=checked+1;
46         end
47         if(checked!=16)
48             $fatal(1,"Incomplete test");
49         $display("LAB1_PASS seg_decoder cases=%0d",checked);
50         $finish;
51     end
52     initial begin
53         #100000;
54         $fatal(1,"Watchdog");
55     end
56 end
57 endmodule

```

TB는 DUT를 인스턴스한다. 그리고 0~F까지 16개 입력을 자동 적용하며 digits 배열에 기대값을 저장한다. 또한, 실제 출력과 기대값을 비교한다. 값이 다르면 \$fatal로

즉시 중단함과 동시에 모두 통과하면 PASS 메시지를 출력한다.

· 코드 커밋 & 도구 버전

```

PS C:\Users\kjh36\Documents\Verilog\lab1_10_seven_segment> git --version
>> python --version
>> code --version
git version 2.55.0.windows.5
Python 3.14.7
1.137.0
645f29cc3176500b4b5762ba887cf2a7f0ffdfdc
arm64

```

```

PS C:\Users\kjh36\Documents\Verilog\lab1_10_seven_segment> iverilog -V
Icarus Verilog version 14.0 (devel) ()

```

```

PS C:\Users\kjh36\Documents\Verilog\lab1_10_seven_segment> vvp -V
Icarus Verilog runtime version 14.0 (devel) ()

```

```

PS C:\Users\kjh36\Documents\Verilog\lab1_10_seven_segment> git branch --show-current
>> git rev-parse HEAD
main
a248b02aa2dec8def8caaf5175f12fc005f8f5c

```

· 입력 조합

TB는 16진수 입력인 '0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F'를 순차적으로 검사한다.

· 기댓값

각 입력마다 digits 배열에 저장된 기대 출력과 비교한다.

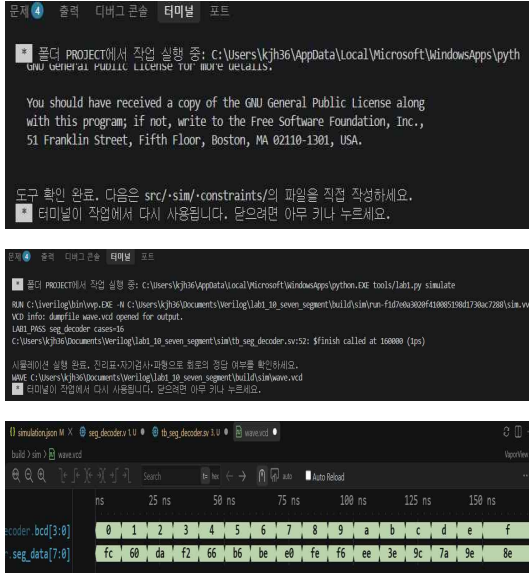
· 자동 비교

기댓값과 실제값이 다르면 시뮬레이션이 '\$fatal'로 인해 즉시 종료된다.

· 종료 조건

모든 테스트에 통과 시 'LAB1_PASS seg_decoder cases=16'을 출력하고 종료한다.

· PASS Log & Wave



· 시간 구간별 값 해석 및 예상값 비교

시간(ns)	bcd[3:0]	seg_data[7:0]	표시 문자	예상값 비교
0~10	0000	FC	0	예상값 FC와 일치
10~20	0001	60	1	예상값 60과 일치
20~30	0010	DA	2	예상값 DA와 일치
30~40	0011	F2	3	예상값 F2와 일치
40~50	0100	66	4	예상값 66과 일치
50~60	0101	B6	5	예상값 B6과 일치
60~70	0110	BE	6	예상값 BE와 일치
70~80	0111	E0	7	예상값 E0과 일치
80~90	1000	FE	8	예상값 FE와 일치
90~100	1001	F6	9	예상값 F6과 일치
100~110	1010	EE	A	예상값 EE와 일치
110~120	1011	3E	b	예상값 3E와 일치
120~130	1100	9C	C	예상값 9C와 일치
130~140	1101	7A	d	예상값 7A와 일치
140~150	1110	9E	E	예상값 9E와 일치
150~160	1111	8E	F	예상값 8E와 일치

0~10ns 구간에서는 bcd=0000이 입력되며 seg_data=FC가 출력되어 숫자 0이 표시되었다.

30~40ns 구간에서는 bcd=0011에 대해 seg_data=F2가 출력되어 숫자 3이 정상적으로 표시되었다.

80~90ns 구간에서는 bcd=1000이 입력되었고 seg_data=FE가 출력되었다. 이는 a~g

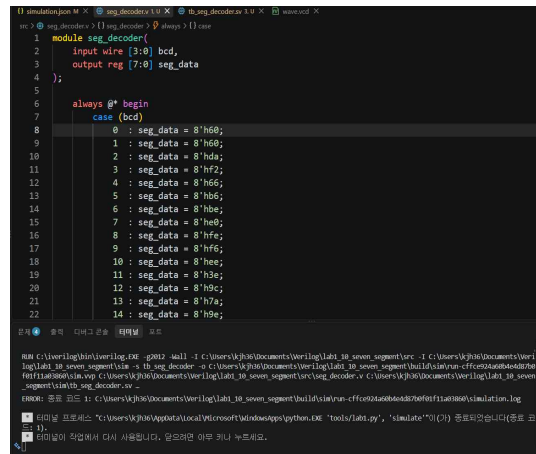
모든 segment가 켜진 상태이므로 숫자 8이 정확히 표시된 것이다.

100~110ns 구간에서는 bcd=1010에 대해 seg_data=EE가 출력되어 문자 A가 표시되었다.

150~160ns 구간에서는 bcd=1111에 대해 seg_data=8E가 출력되어 문자 F가 정상적으로 표시되었다.

· 오류, 수정, 재실행 및 보드에서 확인할 입출력

RTL에서 '0 : seg_data = 8'hFC;'을 '0 : seg_data = 8'h60;'로 바꾼다. 이렇게 되면 입력이 'bcd'일 때 기댓값은 FC이지만 실제값으로 60이 나오게 되어 TB에서 FAIL을 검출한다.



해당 부분을 원래대로 수정한 뒤 재실행하면 정상적으로 PASS가 나오는 것을 알 수 있다. 그리고 보드에서 확인할 입력은 DIP 1 ~ 4 → bcd[3:0]이고 출력은 7-segment → a, b, c, d, e, f, g, dp이다.

11. 참고 문헌

- 전자전기컴퓨터설계II 교안 : LAB1_01 ~ 10_VIVADO
- TechWeb - product - opto electronics
- TechWeb TOP 디바이스 관련 기사 - 광 반도체 - LED 분야